

Лекция 10

1. Прогон программы, демонстрирующей передачу информации от одного процесса к другому через разделяемую память

```
1  #include <windows.h>
2  #include <stdio.h>
3  int main() {
4      HANDLE hMapFile;
5      LPVOID lpMapAddress;
6      HANDLE hFile;
7      char* String;
8      hFile = CreateFile(L"MyFile.txt", GENERIC_READ | GENERIC_WRITE, FILE_SHARE_READ | FILE_SHARE_WRITE,
9                          NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
10     if (hFile == INVALID_HANDLE_VALUE) {
11         printf("Could not open file\n");
12         return 1;
13     }
14     hMapFile = CreateFileMapping(hFile, NULL, PAGE_READWRITE, 0, 256, L"MyFileObject");
15
16     if (hMapFile == NULL) {
17         printf("Could not create file-mapping object.\n");
18         CloseHandle(hFile);
19         return 1;
20     }
21     lpMapAddress = MapViewOfFile(hMapFile, FILE_MAP_ALL_ACCESS, 0, 0, 0);
22
23     if (lpMapAddress == NULL) {
24         printf("Could not map view of file.\n");
25         CloseHandle(hMapFile);
26         CloseHandle(hFile);
27         return 1;
28     }
```

```
29     String = (char*)lpMapAddress;
30     if (String == NULL) {
31         printf("Could not allocate memory for the buffer.\n");
32         UnmapViewOfFile(lpMapAddress);
33         CloseHandle(hMapFile);
34         CloseHandle(hFile);
35         return 1;
36     }
37     int result = _snprintf_s(String, 256, _TRUNCATE, "%s", "Hello, world");
38
39     if (result == -1) {
40         printf("Could not write string to the buffer.\n");
41         UnmapViewOfFile(lpMapAddress);
42         CloseHandle(hMapFile);
43         CloseHandle(hFile);
44         return 1;
45     }
46
47     UnmapViewOfFile(lpMapAddress);
48     CloseHandle(hMapFile);
49     CloseHandle(hFile);
50     return 0;
51 }
```

Результат:

```
Could not open file

C:\Users\Анастасия\source\repos\lecture_10\x64\Debug\lec
Нажмите любую клавишу, чтобы закрыть это окно...
```

2.

```
1  ✓ #include <windows.h>
2  [ #include <stdio.h>
3  ✓ void main(void) {
4      HANDLE hMapFile;
5      LPVOID lpMapAddress;
6      HANDLE hFile;
7      char* String;
8      hFile = CreateFile(L"MyFile.txt", // имя файла
9          GENERIC_READ | GENERIC_WRITE, // файл для чтения и записи
10         FILE_SHARE_READ | FILE_SHARE_WRITE,
11         NULL, // защита по умолчанию
12         OPEN_EXISTING, // файл должен существовать
13         FILE_ATTRIBUTE_NORMAL, // атрибуты файла
14         NULL); // файл атрибутов
15  ✓ if (hFile == INVALID_HANDLE_VALUE)
16      {
17          printf("Could not open file\n"); // process error
18      }
19      hMapFile = OpenFileMapping(FILE_MAP_ALL_ACCESS,
20          FALSE, // описатель не наследуется
21          L"MyFileObject"); // имя объекта проецируемого файла
22      if (hMapFile == NULL) printf("Could not open Filemapping\n");
23      lpMapAddress = MapViewOfFile(hMapFile,
24          FILE_MAP_ALL_ACCESS, // режимы доступа
25          0, 0, // отображение файла с начала
26          0); // отображение целого файла
27      if (lpMapAddress == NULL) printf("Could not map view of file.\n");
28      String = (char*)lpMapAddress;
29      printf("%s\n", String);
30      getchar();
31  }
```

Результат:

```
Could not open file
Could not open Filemapping
Could not map view of file.
L(null)
```

3. Прогон программы, иллюстрирующей увеличение рабочего набора процесса НЕ РАБОТАЕТ.

```

1  ✓ #include <windows.h>
2  | #include <stdio.h>
3  ✓ int main(void)
4  {
5      PVOID pMem = NULL;
6      int nPageSize = 4096;
7      int nPages = 200;
8      long SizeCommit = 0;
9      int i;
10     char* Ptr;
11     SizeCommit = nPages * nPageSize;
12     getchar();
13     pMem = VirtualAlloc(0, SizeCommit, MEM_RESERVE | MEM_COMMIT,
14         PAGE_READWRITE);
15     if (pMem == NULL) printf("VirtualAlloc Error\n");
16     Ptr = (char*)pMem;
17     for (i = 0; i < nPages; i++) Ptr[i * nPageSize] = '0';
18     getchar();
19     pMem = VirtualAlloc(0, SizeCommit, MEM_RESERVE | MEM_COMMIT,
20         PAGE_READWRITE);
21     if (pMem == NULL) printf("VirtualAlloc Error\n");
22     Ptr = (char*)pMem;
23     for (i = 0; i < nPages; i++) Ptr[i * nPageSize] = '0';
24     getchar();
25     pMem = VirtualAlloc(0, SizeCommit, MEM_RESERVE | MEM_COMMIT,
26         PAGE_READWRITE);
27     if (pMem == NULL) printf("VirtualAlloc Error\n");
28     Ptr = (char*)pMem;
29     for (i = 0; i < nPages; i++) Ptr[i * nPageSize] = '0';
30     getchar();
31     pMem = VirtualAlloc(0, SizeCommit, MEM_RESERVE | MEM_COMMIT,
32         PAGE_READWRITE);
33     if (pMem == NULL) printf("VirtualAlloc Error\n");
34     Ptr = (char*)pMem;
35     for (i = 0; i < nPages; i++) Ptr[i * nPageSize] = '0';
36     getchar();
37     return 0;
38 }

```

4. Прогон программы, демонстрирующей блокировку страниц в памяти

```
#include <windows.h>
#include <stdio.h>
void main(void)
{
    PVOID pMem = NULL;
    int nPageSize = 4096;
    int nPages = 400;
    int nPageLock = 100;
    long SizeCommit = 0;
    int i;
    char* Ptr;
    int nMinPages = 200, nMaxPages = 500;
    long dwMinimumWorkingSetSize = 0, dwMaximumWorkingSetSize = 0;
    HANDLE hProcess;
    hProcess = GetCurrentProcess();
    dwMinimumWorkingSetSize = nMinPages * nPageSize;
    dwMaximumWorkingSetSize = nMaxPages * nPageSize;
    i = SetProcessWorkingSetSize(hProcess, dwMinimumWorkingSetSize,
        dwMaximumWorkingSetSize);
    if (i == 0) printf("SetProcessWorkingSetSize Error\n");
    SizeCommit = nPages * nPageSize;
    pMem = VirtualAlloc(0, SizeCommit, MEM_RESERVE | MEM_COMMIT,
        PAGE_READWRITE);
    if (pMem == NULL) printf("VirtualAlloc Error\n");
    Ptr = (char*)pMem;
    for (i = 0; i < nPages; i++) Ptr[i * nPageSize] = '0';
    i = VirtualLock(pMem, nPageLock * nPageSize);
    if (i == 0) printf("VirtualLock Error\n");
    for (i = 0; i < nPages; i++) Ptr[i * nPageSize] = '0';
    VirtualUnlock(pMem, nPageLock * nPageSize);
    VirtualFree(pMem, 0, MEM_RELEASE);
}
```

КОНСОЛЬ ОТЛАДКИ MICROSOFT VISUAL STUDIO

```
C:\Users\Анастасия\source\repos\lecture_10\x64\Debug\lecture_10.
Нажмите любую клавишу, чтобы закрыть это окно...
```

5. Прогон программы, иллюстрирующей отложенное выделение памяти НЕ РАБОТАЕТ

```

1  ~#include <windows.h>
2  ~#include <stdio.h>
3  ~void main(void)
4  {
5      HANDLE hMapFile;
6      LPVOID lpMapAddress;
7      HANDLE hFile;
8      char* String;
9      hFile = CreateFile(L"MyFile.txt", GENERIC_READ | GENERIC_WRITE,
10         FILE_SHARE_READ | FILE_SHARE_WRITE, NULL, OPEN_ALWAYS,
11         FILE_ATTRIBUTE_NORMAL, NULL);
12      if (hFile == INVALID_HANDLE_VALUE) printf("Could not open file\n");
13      hMapFile = CreateFileMapping(hFile, NULL,
14         PAGE_WRITECOPY, // копирование при записи
15         0, 0, L"MyFileObject");
16      if (hMapFile == NULL) printf("Could not create file-mapping object.\n");
17      lpMapAddress = MapViewOfFile(hMapFile,
18         FILE_MAP_COPY, // копирование при записи
19         0, 0, 0);
20      if (lpMapAddress == NULL) printf("Could not map view of file.\n");
21      String = (char*)lpMapAddress;
22      getchar();
23      sprintf(String, "Hello, world");
24      printf("%s\n", String);
25      if (!UnmapViewOfFile(lpMapAddress)) printf("Could not unmap view of file.\n");
26  }

```